Claude Code

A Practical Reference Guide for AI Engineering Work

September 2026

Contents

Overview & How to Use This Guide **7**
The mental model, in one paragraph . . . 7
Which feature do I need? (decision table) . . . 7
Build it when you notice this (don't over-engineer day one) . . . 8
Quick command reference . . . 8
The rule to never confuse . . . 9

01 — Getting Started **10**
What it is . . . 10
Step 1 — Pick a surface and install . . . 10
Terminal (most flexible — start here) . . . 10
VS Code . . . 11
Desktop app . . . 11
Web . . . 11
JetBrains . . . 11
Step 2 — First run . . . 11
Step 3 — Bootstrap your project . . . 11
Choosing a surface — quick guidance . . . 12
Moving a session between surfaces . . . 12

02 — How Claude Code Works **13**
The agentic loop . . . 13
The 5 built-in tool categories . . . 13
Example loop: "fix the failing tests" . . . 14
What Claude can access when you run `claude` . . . 14
Sessions . . . 14
Resume vs Fork . . . 14
Safety: checkpoints . . . 15
Safety: permission modes . . . 15
Working style tips . . . 15

03 — CLAUDE.md, Rules, and Project Setup **17**
The golden rule . . . 17
Project-level files (`.claude/` in your repo root — mostly committed to git) 17
Global-level files (`~/.claude/` — personal, applies to every project) . . . 18
Other files worth knowing about (not shown in the tree above) . . . 18
Step by step: writing a good CLAUDE.md . . . 19
Example CLAUDE.md . . . 19
Path-scoped rules (`.claude/rules/*.md`) . . . 19
settings.json — what actually gets enforced . . . 20
.mcp.json — team-shared external tool connections . . . 21
.worktreeinclude — carry untracked files into new worktrees . . . 21

How layering/precedence works . . . 21
Cleaning up project data . . . 22

04 — Skills and Commands **23**
What a skill is . . . 23
Anatomy . . . 23
Example: SKILL.md . . . 23
Example: checklist.md (a bundled supporting file) . . . 24
Key frontmatter options . . . 24
Passing arguments . . . 25
Skills vs Commands . . . 25
When to build one . . . 25
Context cost . . . 25

05 — Subagents and Agent Teams **26**
Subagent vs Skill (the distinction that matters) . . . 26
Anatomy of a subagent . . . 26
Why subagents actually save context (real numbers) . . . 27
Fork vs fresh subagent . . . 27
Persistent subagent memory . . . 27
Dynamic workflows (“agent teams”) . . . 28
Cross-session messaging . . . 28
When to reach for which . . . 28

06 — Hooks and MCP **29**
Hooks — automation that always fires, no asking . . . 29
Example: auto-format after every edit . . . 29
How hook output reaches Claude (important nuance) . . . 29
When to use one . . . 30
MCP — connecting to external services . . . 30
Example: `.mcp.json` (project root, team-shared) . . . 30
Scopes . . . 30
Deferred tool loading (keeps context cheap) . . . 31
Combination patterns worth remembering . . . 31

07 — Context Window & Prompt Caching **32**
Why this matters . . . 32
Part 1 — What loads before you type anything . . . 32
Part 2 — Visibility ≠ cost (the trap to know about) . . . 32
Part 3 — Prompt caching mechanics . . . 33
The 3 layers (front to back) . . . 33
What invalidates the cache (costs one slow/expensive turn) . . . 33
What’s safe (keeps the cache warm) . . . 34
TTL (cache lifetime) . . . 34
Check your cache performance . . . 34
Practical takeaways . . . 34
Part 4 — What survives `/compact` . . . 35
Part 5 — Levers you control before hitting the wall . . . 35
One fact worth remembering . . . 36

08 — Memory, Sessions, Worktrees & Checkpoints (Deep Dive) 37
CLAUDE.md: the full hierarchy . . . 37
@import syntax . . . 37
Working with other AI tools' config (AGENTS.md, Cursor, Copilot) . 38
Auto memory — the 4 kinds Claude writes for itself . . . 38
Where it lives . . . 38
Sessions . . . 39
Resume vs Branch vs Fork — three different things . . . 39
Name your sessions . . . 39
Session picker shortcuts . . . 39
Exporting a conversation . . . 40
Worktrees — isolated parallel sessions . . . 40
Isolation enforcement . . . 40
Copy untracked files into every new worktree . . . 40
Isolate subagents in their own worktree . . . 40
Cleanup . . . 41
Checkpointing — undoing Claude's work . . . 41

09 — Agents in Parallel (Full Comparison) 42
The master comparison table . . . 42
Choosing which one . . . 43
Agent view — `claude agents` . . . 43
Quick start . . . 43
From your shell . . . 44
From inside a session . . . 44
File isolation . . . 44
Manage from the shell . . . 44
Agent teams (experimental, disabled by default) . . . 45
Subagents vs Agent teams . . . 45
Starting a team . . . 45
Best practices . . . 46
Known limitations . . . 46
Dynamic workflows — script-based orchestration . . . 46
Cross-session messaging . . . 47

10 — MCP Deep Dive 48
What people actually use it for . . . 48
Adding a server — 4 transport types . . . 48
1. Remote HTTP (recommended for cloud services) . . . 48
2. Remote SSE (deprecated — use HTTP where available) . . . 49
3. Local stdio (runs as a process on your machine) . . . 49
4. Remote WebSocket (servers that push events unprompted) . . . 49
Scopes — who sees the server . . . 49
Managing servers . . . 50
Authentication . . . 50
Environment variable expansion in `.mcp.json` . . . 51
Reference MCP resources with @-mentions . . . 51
MCP prompts as commands . . . 52
Tool search — keeps context cheap as you add servers . . . 52
Output limits . . . 52
Worked examples . . . 53

Using Claude Code itself as an MCP server . . . 53

11 — Skills Deep Dive **54**
Decision rule: when to write one . . . 54
Anatomy (recap + more detail) . . . 54
A full worked example . . . 54
Every frontmatter field . . . 55
Bundled skills (ship with Claude Code, available every session) . . . 55
Bundling reference material . . . 56
Precedence when names collide . . . 56
Skills vs Commands (recap) . . . 56
Skills in the Agent SDK . . . 56
If your YAML frontmatter is malformed . . . 56

12 — Hooks Reference **57**
The full event list, by cadence . . . 57
Exit code behavior (the part that trips people up) . . . 57
Common input JSON fields (every hook receives these) . . . 58
Environment variables available to command hooks . . . 58
Practical example: block Claude from reading `.env` . . . 59
Practical example: auto-format after every edit . . . 59
Re-inject context after compaction . . . 60
Enforcing quality gates on agent teams . . . 60
Hooks vs Skills — which one . . . 60

13 — Permissions & Security **61**
The golden rule (worth repeating) . . . 61
Rule syntax . . . 61
Evaluation order — this is the part people get wrong . . . 61
Where rules live (4 scopes) . . . 62
Permission modes (recap + detail) . . . 62
Matching a specific parameter, not just the tool . . . 63
Sandboxing (brief) . . . 63
Security basics worth internalizing . . . 63

14 — Settings, CLI & Commands Reference **64**
Settings precedence (highest wins) . . . 64
A realistic full settings.json . . . 64
CLI flags worth knowing . . . 65
Built-in slash commands (grouped) . . . 65
Environment variables worth knowing . . . 66
Where Claude Code stores things (recap) . . . 67

15 — Model Config & Costs **68**
Choosing a model . . . 68
Effort levels . . . 68
Context windows . . . 69
Fast mode . . . 69
The advisor tool . . . 69
Managing costs . . . 69
The numbers (enterprise average, for context) . . . 69

Reduce token usage — the concrete levers . . . 70
Agent team costs specifically (see file 09) . . . 70

16 — Tools, Plugins & Platform Integrations 71
Built-in tools (beyond the 5 categories in file 02) . . . 71
Plugins — bundling everything into one installable unit . . . 71
Discovering plugins . . . 72
Platform integrations (condensed reference) . . . 72
Glossary — terms worth knowing cold . . . 73

17 — Best Practices & the Agent SDK 75
Best practices (condensed) . . . 75
Common workflow patterns (from Anthropic's own guide) . . . 75
Large codebases / monorepos . . . 76
The Agent SDK — building your own products with Claude Code's engine 76
Install . . . 76
Minimal working example (Python) . . . 76
Minimal working example (TypeScript) . . . 77
What carries over from Claude Code automatically . . . 77
Key building blocks . . . 78
Permission evaluation order in the SDK (same spirit as file 13, SDK-specific) . . . 78
Error handling (Python) . . . 79
Where to go deeper . . . 79

Overview & How to Use This Guide

Compiled from Anthropic's official Claude Code documentation, organized by topic so you can look things up while you're actually building — not re-read everything from scratch.

> Claude Code moves fast. These are accurate as of the docs pulled together in September 2026 — if something feels off in six months, cross-check at `docs.claude.com`.

The mental model, in one paragraph

Claude Code = a **model** (the reasoner) wrapped in a **harness** (tools + context management + execution environment). Every task runs an **agentic loop**: gather context → take action → verify → repeat, adapting to what it learns as it goes. Everything you configure — CLAUDE.md, skills, subagents, hooks, MCP — falls into one of three buckets:

- **Always-on context** — loaded every session (CLAUDE.md, rules without path scoping)
- **On-demand capability** — invoked when needed (skills, subagents)
- **Background automation** — fires on events, no asking (hooks)

Deciding which bucket a given piece of knowledge belongs in controls both **token cost** and **reliability**.

Which feature do I need? (decision table)

Feature	What it does	When to use it
CLAUDE.md	Persistent context, loaded every session	Project conventions, "always do X" rules
Skill	Reusable instructions/workflow, invocable	Reference docs, repeatable procedures

Feature	What it does	When to use it
Subagent	Isolated worker, own context window	Multi-file research, parallel/specialized work
Dynamic workflow	Script running many subagents	Work that outgrows a handful of subagents
Code intelligence	Language-server navigation	Typed languages, large codebases
MCP	Connects to external services	External data or actions (DB, Slack, APIs)
Hook	Auto-runs on lifecycle events	Something that must happen every time, no asking
Plugin	Bundles the above, shareable	Reuse across repos / distribute to others

Build it when you notice this (don’t over-engineer day one)

You notice…	Add…
Claude gets a convention wrong twice	A CLAUDE.md entry
You keep typing the same starter prompt	A user-invocable skill
You’ve pasted the same playbook 3 times	Capture it as a skill
You keep copying data from a tab Claude can’t see	An MCP server
Claude reads many files to find one symbol	A code intelligence plugin
A side task floods context with junk you won’t reuse	Route it through a subagent
You want something automatic, no asking	A hook
A second repo needs the same setup	Package it as a plugin

Quick command reference

Command	Does
`/init`	Auto-generates a starter CLAUDE.md
`/doctor`	Diagnoses setup issues; suggests CLAUDE.md trims
`/memory`	Open/edit CLAUDE.md from inside a session
`/model`	Switch model mid-session
`/context`	Live breakdown of what's using your context window
`/compact [focus]`	Summarize conversation to free space
`/clear`	Full reset — use between unrelated tasks
`/rewind`	Jump back to an earlier turn (cheaper than compact)
`/recap`	Terminal-only summary, doesn't touch the cache
`/usage`	Session cost + cache hit-rate breakdown
`/permissions`	View/edit tool permission rules
`/branch` or `--fork-session`	Copy a session into a new one
`--continue` / `--resume`	Reopen a session, same ID
`/workflows`	Save a dynamic workflow script
`/plugin`, `/reload-plugins`	Manage plugins
`/autocompact <tokens>`	Set your own auto-compact threshold
`claude project purge <path>`	Delete all stored state for a project

The rule to never confuse

- **CLAUDE.md / rules** = guidance Claude *reads* — it can still deviate.
- **settings.json permissions/hooks** = *enforced* — happens regardless of what Claude "wants."

01 — Getting Started

What it is

Claude Code is an agentic coding tool: it reads your codebase, edits files, runs terminal commands, and integrates with your dev tools. It's not autocomplete — it can actually execute, verify, and iterate on its own work.

Step 1 — Pick a surface and install

Terminal (most flexible — start here)

macOS / Linux / WSL:

```
curl -fsSL https://claude.ai/install.sh | bash
```

Windows PowerShell:

```
irm https://claude.ai/install.ps1 | iex
```

Windows CMD:

```
curl -fsSL https://claude.ai/install.cmd -o install.cmd && install.cmd && del
↪  install.cmd
```

Alternative installers:

```
brew install --cask claude-code       # Homebrew (stable channel)
winget install Anthropic.ClaudeCode   # WinGet
```

> Native installs auto-update in the background. Homebrew/WinGet installs need a manual `brew upgrade claude-code` / `winget upgrade Anthropic.ClaudeCode`.

VS Code

Search “Claude Code” in the Extensions view (`Cmd+Shift+X` / `Ctrl+Shift+X`) → install → open Command Palette → type “Claude Code” → **Open in New Tab**.

Desktop app

Download for macOS or Windows, sign in, click the **Code** tab. Bundles the CLI — no separate install needed. Requires a paid subscription.

Web

No install — go to `claude.ai/code`. Good for repos you don’t have locally, or long tasks you want to check back on later.

JetBrains

Install the plugin from the JetBrains Marketplace. Requires the CLI installed separately too.

Step 2 — First run

```
cd your-project
claude
```

First time: you’ll be prompted to log in (or, if you’ve set `ANTHROPIC_API_KEY`, asked to approve that key instead).

Step 3 — Bootstrap your project

Inside a session, run: - **`/init`** — walks you through creating a starter `CLAUDE.md` for this project - **`/doctor`** — a setup checkup that diagnoses install/config issues; later, once your CLAUDE.md has grown, it also suggests trims

Choosing a surface — quick guidance

If you want…	Use
Full control, scripting, CI pipelines	Terminal
Inline diffs while coding	VS Code / JetBrains
Visual diff review, multiple sessions side by side	Desktop app
No local setup, or a repo you don’t have locally	Web

All surfaces share the same engine — same CLAUDE.md, same settings, same MCP servers. Switch anytime; nothing is locked to one surface.

Moving a session between surfaces

I want to…	Command
Push a local session to web/mobile	`claude --cloud`
Pull a cloud session back to terminal	`claude --teleport`
Move a terminal session into the visual diff app	`/desktop`

(Teleport and `/desktop` require a claude.ai subscription.)

02 — How Claude Code Works

The agentic loop

Every task cycles through three phases, blended together and repeated until done:

```
gather context → take action → verify results → (repeat)
```

Claude decides how much of each phase a task needs. A question might only need context-gathering. A bug fix cycles through all three repeatedly. You can interrupt at any point to redirect it.

Two components power this: - **Model** — the reasoning. Sonnet handles most coding tasks well; Opus gives stronger reasoning for complex architectural decisions. Switch with `/model` mid-session, or start with `claude --model <name>`. - **Tools** — the acting. Claude Code is the "agentic harness" — it supplies the tools, context management, and execution environment around the model.

The 5 built-in tool categories

Category	Claude can…
File operations	Read, edit, create, rename, reorganize files
Search	Find files by pattern, regex search, explore codebases
Execution	Run shell commands, start servers, run tests, use git
Web	Search the web, fetch docs, look up error messages
Code intelligence	See type errors after edits, jump to definitions (needs a plugin)

Example loop: “fix the failing tests”

1. Run the test suite → see what’s failing
2. Read the error output
3. Search for relevant source files
4. Read those files for context
5. Edit to fix the issue
6. Re-run tests to verify

Each tool result feeds directly into the next decision — that’s the loop in action.

What Claude can access when you run `claude`

- Your project files (plus others, with permission)
- Your terminal — anything you could run from the command line
- Your git state — current branch, uncommitted changes, recent commits
- Your `CLAUDE.md`
- Auto memory (first 200 lines / 25KB of `MEMORY.md`, whichever comes first)
- Configured extensions: MCP servers, skills, subagents, Claude in Chrome

Sessions

- **Independent** — each new session starts with a fresh context window. Only CLAUDE.md and auto memory carry over automatically.
- **Tied to directory, not branch** — switching git branches keeps your conversation, just shows the new branch’s files.
- Stored as plaintext JSONL under `~/.claude/projects/` — this is what powers rewind/resume/fork.
- Want true parallel sessions? Use **git worktrees** — separate directories per branch.

Resume vs Fork

	Same session ID	New session ID
Resume (`--continue` / `--resume`)	Yes — appends to existing conversation	
Fork (`--fork-session` / `/branch`)		Yes — copies history, original untouched

Safety: checkpoints

File edits are automatically snapshotted before every change. Press `Esc` twice to rewind to a previous state, or just ask Claude to undo. Checkpoints are separate from git and remain available even after resuming a conversation.

> Checkpoints only cover **file changes** — not database writes, API calls, or deployments. You control those with your permission mode and permission rules instead.

Safety: permission modes

Cycle through with `Shift+Tab`:

Mode	Behavior
Auto	A classifier reviews most actions in the background, only blocking risky ones (default starting mode on Pro/Max/Team)
Manual	Claude asks before every file edit and shell command
Accept edits	Edits + safe filesystem commands (`mkdir`, `mv`) run without asking; other commands still ask
Plan	Claude explores and proposes a plan without editing your source files

You can also pre-allow specific commands in `.claude/settings.json` (e.g. `npm test`, `git status`) so Claude stops asking about those — see the *CLAUDE.md, Rules, and Project Setup* chapter.

Working style tips

- **It's conversational.** Don't over-engineer the first prompt. Start with intent, refine as Claude works.

```
Fix the login bug
[Claude investigates, tries something]
That's not quite right — the issue is in the session handling.
[Claude adjusts approach]
```

- **Interrupt and steer.** Press `Esc` to cancel the running tool call immediately. Or type a correction and hit `Enter` without stopping — Claude reads it as soon as the current action finishes.
- **Delegate, don't dictate.** Give context + a goal, the way you'd brief a capable colleague: > "The checkout flow is broken for users with expired cards. The relevant code is in `src/payments/`. Can you investigate and fix it?"

  You don't need to specify which files to read or which commands to run — Claude figures that out.

03 — CLAUDE.md, Rules, and Project Setup

The golden rule

- **CLAUDE.md / rules/** = guidance Claude *reads*. It can still deviate.
- **settings.json permissions/hooks** = *enforced*, regardless of what Claude "wants."

Project-level files (`.claude/` in your repo root — mostly committed to git)

File / folder	Purpose	Committed?
`CLAUDE.md` (project root)	Instructions loaded every session	Yes
`.mcp.json` (project root)	Team-shared MCP servers	Yes
`.worktreeinclude` (project root)	Gitignored files to copy into new worktrees	Yes
`.claude/settings.json`	Permissions, hooks, model default, env vars — **enforced**	Yes
`.claude/settings.local.json`	Your personal overrides	No (auto-gitignored)
`.claude/rules/*.md`	Topic-scoped instructions, optionally path-gated	Yes
`.claude/skills/<name>/SKILL.md`	Reusable invocable workflow	Yes
`.claude/commands/*.md`	Legacy single-file version of skills	Yes
`.claude/agents/*.md`	Subagent definitions	Yes

File / folder	Purpose	Committed?
`.claude/workflows/*.js`	Dynamic workflow scripts	Yes
`.claude/agent-memory/<name>/MEMORY.md`	A subagent's own persistent memory	Yes
`.claude/output-styles/*.md`	Shared team output styles (rare)	Yes

Global-level files (`~/.claude/` — personal, applies to every project)

Same folder names mirrored (`CLAUDE.md`, `settings.json`, `rules/`, `skills/`, `commands/`, `agents/`, `workflows/`, `output-styles/`) — all personal, never committed. Plus:

- `~/.claude.json` — app state: theme, OAuth session, personal MCP servers, per-project trust decisions
- `~/.claude/keybindings.json` — custom shortcuts
- `~/.claude/themes/*.json` — custom color themes
- `~/.claude/projects/<project>/memory/` — **auto memory**: Claude's own notes to itself, written automatically as it works

Other files worth knowing about (not shown in the tree above)

File	Location	Purpose
`managed-settings.json`	System-level, varies by OS	Enterprise-enforced settings you can't override
`CLAUDE.local.md`	Project root	Your private per-project preferences, loaded alongside CLAUDE.md — create manually and add to `.gitignore`

Step by step: writing a good CLAUDE.md

1. Run `/init` to generate a starting point, or create it by hand at your project root.
2. Include: build/test/lint commands, stack/framework conventions, naming rules, file-layout conventions.
3. Keep it **under 200 lines**. Longer files still load in full but tend to reduce adherence.
4. Move anything only needed *sometimes* into a skill or a path-scoped rule (below) — CLAUDE.md loads into **every** session regardless of what you're actually doing.
5. Edit anytime with `/memory`. Run `/doctor` for trim proposals once it's grown.

Example CLAUDE.md

```
# Project conventions

## Commands
- Build: `npm run build`
- Test: `npm test`
- Lint: `npm run lint`

## Stack
- TypeScript with strict mode
- React 19, functional components only

## Rules
- Named exports, never default exports
- Tests live next to source: `foo.ts` -> `foo.test.ts`
- All API routes return `{ data, error }` shape
```

> **Note: Editing CLAUDE.md mid-session doesn't apply immediately.** Claude keeps working with the version loaded at session start. The new content loads on the next `/clear`, `/compact`, or restart.

Path-scoped rules (`.claude/rules/*.md`)

Use when a rule only matters for certain files — it loads automatically the moment Claude reads a matching file, and stays out of context the rest of the time.

```
---
paths:
  - "**/*.test.ts"
  - "**/*.test.tsx"


# Testing Rules

- Use descriptive test names: "should [expected] when [condition]"
- Mock external dependencies, not internal modules
- Clean up side effects in afterEach
```

A rule **without** `paths:` frontmatter behaves like CLAUDE.md — loads at session start for every conversation.

settings.json — what actually gets enforced

```
{
  "permissions": {
    "allow": ["Bash(npm test *)", "Bash(npm run *)"],
    "deny": ["Bash(rm -rf *)"]
  },
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
      }]
    }]
  }
}
```

- Bash permission patterns support wildcards: `Bash(npm test *)` matches anything starting with `npm test`.
- `settings.local.json` uses the same schema — for personal overrides not shared with the team.

.mcp.json — team-shared external tool connections

```
{
  "mcpServers": {
    "notion": {
      "command": "npx",
      "args": ["-y", "@notionhq/notion-mcp-server"],
      "env": { "NOTION_TOKEN": "${NOTION_TOKEN}" }
    }
  }
}
```

Use `${VAR}` references for secrets — never hardcode tokens; the value is read from your shell environment when the server starts. For servers only *you* need (not the whole team), run `claude mcp add --scope user` instead, which writes to `~/.claude.json`.

.worktreeinclude — carry untracked files into new worktrees

```
# Local environment
.env
.env.local

# API credentials
config/secrets.json
```

Worktrees are fresh checkouts — untracked files like `.env` are missing by default. This copies gitignored files matching these patterns into every new worktree Claude creates.

How layering/precedence works

Feature	Rule
CLAUDE.md	**Additive** — all levels (global + project + nested) load together; more specific instructions typically win on conflict
Skills / subagents	**Override by name** — managed > user > project
MCP servers	**Override by name** — local > project > user
Hooks	**Merge** — all matching hooks fire regardless of source

Cleaning up project data

```
claude project purge ~/work/my-repo --dry-run   # preview what would be deleted
claude project purge ~/work/my-repo             # delete (asks for confirmation)
claude project purge ~/work/my-repo --yes       # skip confirmation, for scripts
```

Deletes transcripts, auto memory, per-session task/debug/file-history data, and matching prompt history for that project. Never deletes `~/.claude.json`, `~/.claude/settings.json`, or `~/.claude/plugins/` — those hold your auth, preferences, and installed plugins.

> **Note:** Transcripts are **plaintext and unencrypted**. If a tool reads a `.env` file, that content gets written to disk as part of the session log. Lower `cleanupPeriodDays` (default 30) in settings if that matters for your setup.

04 — Skills and Commands

What a skill is

A markdown file (plus optional supporting files) containing knowledge, instructions, or a workflow. You invoke it with `/name`, or Claude auto-invokes it when relevant based on its description.

Anatomy

```
.claude/skills/security-review/
├── SKILL.md          ← entrypoint: trigger, invocability, instructions
└── checklist.md      ← supporting file, read on demand
```

Example: SKILL.md

```
---
description: Reviews code changes for security vulnerabilities, authentication
 ↪  gaps, and injection risks
disable-model-invocation: true
argument-hint: <branch-or-path>


## Diff to review

!`git diff $ARGUMENTS`

Audit the changes above for:

1. Injection vulnerabilities (SQL, XSS, command)
2. Authentication and authorization gaps
3. Hardcoded secrets or credentials
```

```
Use checklist.md in this skill directory for the full review checklist.

Report findings with severity ratings and remediation steps.
```

Example: checklist.md (a bundled supporting file)

```
# Security Review Checklist

## Input Validation
- [ ] All user input sanitized before DB queries
- [ ] File upload MIME types validated
- [ ] Path traversal prevented on file operations

## Authentication
- [ ] JWT tokens expire after 24 hours
- [ ] API keys stored in environment variables
- [ ] Passwords hashed with bcrypt or argon2
```

Claude knows the skill directory path, so it can read `checklist.md` on its own once the SKILL.md instructions mention it.

Key frontmatter options

Field	Effect
`description`	Determines when Claude auto-invokes it
`disable-model-invocation: true`	Only *you* can trigger it via `/name` — Claude never invokes it on its own. Use for anything with side effects (deploy, commit, send messages)
`user-invocable: false`	Hidden from the `/` menu — Claude can still invoke it, you can't manually
`argument-hint`	Shown in the `/` menu as a hint for what to pass

Passing arguments

`/deploy staging` → passes `"staging"` as `$ARGUMENTS`. Use `$0`, `$1`, etc. for positional access to individual arguments.

Skills vs Commands

Commands (`.claude/commands/*.md`) are the older, single-file mechanism — same `/name` invocation, but no bundled supporting files. **New workflows should use skills.** If a skill and a command share a name, the skill wins.

When to build one

You notice…	Action
You keep typing the same starter prompt	Make it a user-invocable skill
You've pasted the same playbook a 3rd time	Capture it as a skill
You have reference material you consult often	Build a reference skill (loaded as knowledge, not run as an action)

Skills come in two flavors: - **Reference** — knowledge Claude uses throughout a session (e.g. your API style guide) - **Action** — tells Claude to *do* something specific (e.g. `/deploy`)

Context cost

Only the **description** loads at session start (so Claude knows what's available). Full content loads only when the skill is actually used. Set `disable-model-invocation: true` to keep even the description out of context until you invoke it manually — the right move for anything with side effects.

05 — Subagents and Agent Teams

Subagent vs Skill (the distinction that matters)

	Skill	Subagent
What it is	Reusable instructions/knowledge	Isolated worker, own context window
Key benefit	Shareable across contexts	Context isolation — only a summary returns
Context cost	Adds to your main window	Separate window entirely
Best for	Reference docs, repeatable workflows	Multi-file research, parallel work, specialized roles

They combine: a subagent can preload skills (`skills:` frontmatter field); a skill can run in isolated context via `context: fork`.

Anatomy of a subagent

`.claude/agents/code-reviewer.md`:

```
---
name: code-reviewer
description: Reviews code for correctness, security, and maintainability
tools: Read, Grep, Glob
---

You are a senior code reviewer. Review for:

1. Correctness: logic errors, edge cases, null handling
2. Security: injection, auth bypass, data exposure
```

```
3. Maintainability: naming, complexity, duplication

Every finding must include a concrete fix.
```

- `description` → tells Claude when to auto-delegate to it
- `tools:` → restricts what it can do (here: read-only — it can inspect but never edit)
- The body → becomes the subagent's system prompt
- Type `@` in the CLI and pick it from autocomplete to delegate directly

Why subagents actually save context (real numbers)

In a documented example session: a subagent was asked to research session-timeout handling. It read `session.ts` (2,200 tokens), `timeouts.ts` (800 tokens), and `config/*.ts` (3,100 tokens) — **6,100 tokens total**. Only its final summary — **420 tokens** — came back to the main conversation.

That gap *is* the entire value of a subagent: large research happens in isolation; you only pay for the conclusion.

Fork vs fresh subagent

- **Fresh subagent** (default) — starts with its own system prompt + CLAUDE.md, **no conversation history**, no main-session auto memory.
- **Fork** (`context: fork` on a skill, or an explicit fork) — inherits the parent's system prompt, tools, and full conversation history so far.

Persistent subagent memory

Add `memory:` to a subagent's frontmatter:

Value	Writes to	Scope
`memory: project`	`.claude/agent-memory/<name>/MEMORY.md`	Shared with your team (committed)
`memory: local`	`.claude/agent-memory-local/`	Kept out of version control

Value	Writes to	Scope
`memory: user`	`~/.claude/agent-memory/`	Cross-project, personal

This is separate from your main session's auto memory — each subagent reads and writes only its own file.

Dynamic workflows ("agent teams")

When a task outgrows a handful of subagents, a **dynamic workflow** is a script (`.claude/workflows/*.js`) that spawns and coordinates *many* subagents from one script, returning one final result. Save a run as one with `/workflows` → `s`. Each saved file becomes its own `/<name>` command.

Example use case: auditing a whole codebase, with a second set of agents cross-checking the first set's findings.

Cross-session messaging

One of your *own* running sessions can pass a message to another — useful when two sessions you're running yourself depend on each other's findings mid-task (e.g. one session warns another that a change it made will break something the other is building on).

When to reach for which

You notice…	Use
A side task would flood your context with junk you won't reference again	Subagent
The task needs several specialized roles working at once	Dynamic workflow ("agent team")
A skill needs to spawn parallel work	Skill + Subagent combo — e.g. `/audit` kicks off security + performance + style subagents in isolated context

06 — Hooks and MCP

Hooks — automation that always fires, no asking

A hook runs your script (or an HTTP request / MCP tool call / prompt / subagent) automatically at a lifecycle event — before or after a tool call, at session start, and so on. Defined in `settings.json`, so it's **enforced**, not just suggested.

Example: auto-format after every edit

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
      }]
    }]
  }
}
```

How hook output reaches Claude (important nuance)

- Plain `stdout` on exit code `0` → **does not** reach Claude's context. Goes to the debug log only.
- JSON output with `hookSpecificOutput.additionalContext` → **does** reach Claude's context.
- For `PostToolUse` hooks, exit code `2` surfaces stderr as an error to Claude, but can't block the action — the tool already ran.
- Output over 10,000 characters is saved to a file — Claude gets a preview plus the file path instead of the full text.

When to use one

Something must happen **every time** a matching event occurs, with no room for Claude to skip it. Examples: linting before every commit, sending a Slack notification after editing a critical file (hook + MCP combo, below).

MCP — connecting to external services

MCP (Model Context Protocol) is how Claude Code reaches outside your local files: databases, Slack, Jira, Google Drive, browsers, or your own custom tooling.

Example: `.mcp.json` (project root, team-shared)

```
{
  "mcpServers": {
    "notion": {
      "command": "npx",
      "args": ["-y", "@notionhq/notion-mcp-server"],
      "env": { "NOTION_TOKEN": "${NOTION_TOKEN}" }
    }
  }
}
```

Use `${VAR}` for secrets — the token is read from your shell environment, never written into the file itself.

Scopes

Scope	Lives in	Shared with team?
Project	`.mcp.json` at repo root	Yes (committed)
Personal	`~/.claude.json` (via `claude mcp add --scope user`)	No

Deferred tool loading (keeps context cheap)

By default, MCP tool *names* load at session start, but full tool *schemas* stay deferred — Claude loads a specific schema on demand via tool search only when a task actually needs it. This is why connecting several MCP servers doesn't immediately blow up your context window.

Combination patterns worth remembering

Pattern	How it works
Skill + MCP	MCP makes the connection; a skill teaches Claude how to use it well (e.g. MCP connects to your DB, a skill documents your schema and query patterns)
Hook + MCP	A hook triggers an external action through MCP (e.g. a post-edit hook sends a Slack notification)

07 — Context Window & Prompt Caching

Why this matters

Claude Code re-sends your **entire conversation** on every turn (models have no memory between API calls). Two mechanisms control what that costs you: **prompt caching** (avoids reprocessing unchanged content) and **context management** (compaction, subagents, on-demand skills — keeps the window from overflowing). Get both right and sessions stay fast and cheap; ignore them and you'll hit slow, expensive turns without knowing why.

Part 1 — What loads before you type anything

In order, all invisible in your terminal: 1. System prompt 2. Auto memory (`MEMORY.md`) 3. Environment info (working dir, platform, shell, OS, git branch/status) 4. MCP tool names (schemas deferred) 5. Skill descriptions 6. Global `~/.claude/CLAUDE.md` 7. Project `CLAUDE.md`

All of this consumes tokens before your first prompt — which is exactly why keeping CLAUDE.md lean matters.

Part 2 — Visibility ≠ cost (the trap to know about)

What you *see* in the terminal and what enters *context* are different things:

Action	What you see	What actually enters context
File read	One-liner: "Read auth.ts"	Full file content (often 1,000–3,000+ tokens)
Path-scoped rule	"Loaded .claude/rules/x.md"	Full rule content, silently

Action	What you see	What actually enters context
Hook	Nothing (unless it returns `additionalContext`)	Only the `additionalContext` JSON field
User-invoked skill (`disable-model-invocation: true`)	Nothing until invoked	**Zero cost until you invoke it**

Takeaway: be specific in prompts ("fix the bug in auth.ts") so Claude reads fewer files. For research-heavy tasks, delegate to a subagent instead (see *Subagents and Agent Teams*).

Part 3 — Prompt caching mechanics

The API matches from the **start** of each request (the "prefix"). If anything early in that prefix changes, everything after it recomputes. There's no partial or per-file caching.

The 3 layers (front to back)

Layer	Content	Changes when
System prompt	Core instructions, tool defs	Tool set changes
Project context	CLAUDE.md, auto memory	Session start, `/clear`, `/compact`
Conversation	Messages, tool results	Every turn

What invalidates the cache (costs one slow/expensive turn)

- Switching models (`/model`) — full recompute, each model has its own cache
- Changing effort level (exception: Fable 5.1 via subscription/API key keeps the cache)
- Turning on fast mode (one-time cost, only first use per session)
- Connecting/disconnecting an MCP server *if* its tools are loaded into the prefix (not deferred)
- Enabling/disabling a plugin *if* it adds MCP servers loaded into the prefix

- Denying an entire tool (e.g. blocking `Bash` globally) — scoped rules like `Bash(rm *)` are safe
- `/compact` (intentional — that's the whole point)
- Accumulating too many images (old ones get dropped in batches)
- Upgrading Claude Code

What's safe (keeps the cache warm)

- Editing files in your repo
- **Editing CLAUDE.md mid-session** — cache stays warm, but the edit itself doesn't apply until `/clear`/`/compact`/restart
- Changing permission mode
- Changing output style
- Invoking skills/commands
- `/recap`, `/rewind`
- Spawning a subagent (gets its own separate cache)

TTL (cache lifetime)

Billing	Main conversation	Everything else
Claude subscription, within plan usage	1 hour	5 min (server-controlled helpers get 1 hr)
Usage credits / API key / cloud provider	5 min	5 min

Set your own with the `promptCacheTtl` setting or `CLAUDE_CODE_PROMPT_CACHE_TTL` env var (`5m` or `1h` only).

Check your cache performance

Run `/usage` → look for the "Prompt cache (main)" line: hit ratio, miss count, and often the likely cause of the last miss.

Practical takeaways

1. Pick model + effort level once at the start of a session — don't toggle mid-task.
2. Save `/compact` for natural breakpoints between tasks, not mid-task.

3. If you edited CLAUDE.md and nothing changed — that's expected; run `/clear` or restart.

Part 4 — What survives `/compact`

Content	After compaction
System prompt, output style	Still applies
Project CLAUDE.md, unscoped rules	Re-injected from disk
Auto memory	Re-injected from disk
Plan-mode plan	Re-injected from disk
Path-scoped rules	Reload only when Claude re-reads a matching file
Files read/edited	Only up to **5** most-recently-modified are re-read
Invoked skill bodies	Re-injected, capped at 5,000 tokens/skill, 25,000 total — oldest dropped first
Hook-added context	Summarized away with everything else
Skill **descriptions** (the index)	Does **NOT** reload — only skills you actually invoked come back

> **Tip:** Put your most important skill instructions **near the top** of `SKILL.md` — truncation keeps the start, not the end.

Part 5 — Levers you control before hitting the wall

Lever	Use it for
`/compact focus on X`	Steer what the summary keeps instead of letting the automatic pass guess
`/rewind` → "Summarize from/up to here"	Compact only part of the conversation
`/autocompact 500k`	Set your own threshold instead of the default
`/clear`	Full reset between unrelated tasks — cheaper than compacting
Subagent delegation	Keeps large research reads out of your main window entirely

Lever	Use it for
`/context`	Live breakdown of what’s using your context right now

One fact worth remembering

Sonnet 5 (the current default model) runs a **1M token context window natively** — no `[1m]` variant to select, unlike some other models.

08 — Memory, Sessions, Worktrees & Checkpoints (Deep Dive)

CLAUDE.md: the full hierarchy

Four scopes, loaded in this order (broadest → most specific — later ones effectively take precedence in a conflict):

Scope	Location	Shared with
Managed policy	OS-specific path (e.g. `/etc/claude-code/CLAUDE.md`)	Everyone in the org, enforced by IT
User	`~/.claude/CLAUDE.md`	Just you, every project
Project	`./CLAUDE.md` or `./.claude/CLAUDE.md`	Team, via git
Local	`./CLAUDE.local.md`	Just you, this project only — gitignore it

All discovered files are **concatenated**, not merged/overridden — content closer to your working directory is read last (i.e., freshest in context). Nested `CLAUDE.md`/`CLAUDE.local.md` in subdirectories load on demand, only when Claude reads a file in that subdirectory.

@import syntax

Pull other files into CLAUDE.md instead of duplicating content:

```
See @README for project overview and @package.json for available npm commands.

# Additional Instructions
- git workflow @docs/git-instructions.md
```

- Relative paths resolve relative to the file doing the importing, not your cwd.

- Recursive imports up to 4 hops deep.
- Wrap a path in backticks (`` `@README` ``) to mention it *without* importing it.
- An import that resolves **outside** your working directory triggers a one-time approval dialog (protects you from a malicious committed file). Decline once and it stays disabled.
- Imported files still load fully at launch — imports help organization, not context size.

Working with other AI tools' config (AGENTS.md, Cursor, Copilot)

Claude Code reads `CLAUDE.md`, not `AGENTS.md`. To share instructions across tools:

```
@AGENTS.md

## Claude Code
Use plan mode for changes under `src/billing/`.
```

Or symlink: `ln -s AGENTS.md CLAUDE.md`. Running `/init` also auto-imports relevant parts of `.cursor/rules/`, `.cursorrules`, and `.github/copilot-instructions.md`. Run `/import` to bring in a supported agent's full config (MCP servers, commands, subagents, skills too).

Auto memory — the 4 kinds Claude writes for itself

Claude classifies each memory it saves under a `type` in frontmatter: | Type | Captures | |—|—| | `user` | Your role, expertise, working preferences | | `feedback` | Corrections you gave, approaches you confirmed | | `project` | Ongoing work/deadlines/decisions not derivable from code | | `reference` | Where to find things outside the project (issue tracker, dashboard) |

Claude **skips** anything derivable from the codebase (architecture, file paths) and anything CLAUDE.md already says. It doesn't save every session — only what seems useful later.

Where it lives

```
~/.claude/projects/<project>/memory/
├── MEMORY.md          # Index, loaded every session (first 200 lines / 25KB)
```

```
├── user_role.md        # Topic file, read on demand only
├── feedback_testing.md
└── ...
```

Auto memory is **machine-local**, shared across all worktrees of the same repo, never synced across machines. Toggle with `/memory`, or per-project: `{"autoMemoryEnabled": false}` in settings.json.

Sessions

Resume vs Branch vs Fork — three different things

Command	Result
`claude --continue`	Reopens most recent conversation, same ID
`claude --resume`	Opens interactive session picker
`claude --resume <name-or-id>`	Resumes directly
`/branch <name>` (in-session)	Copies conversation to a **new session**, leaves original untouched
`claude --resume --fork-session`	Same as `/branch` from the CLI

Name your sessions

```
claude -n auth-refactor        # at startup
```

or `/rename auth-refactor` mid-session. Named sessions are easier to find later in `claude --resume <name>` or the picker (`Ctrl+R` renames from the picker).

Session picker shortcuts

Key	Action
`↑/↓`	Navigate
`Enter`	Resume
`Space`	Preview without resuming
`Ctrl+A`	Show sessions from every project on the machine

Key	Action
`Ctrl+W`	Show sessions from every worktree of this repo
`Ctrl+B`	Filter to current git branch

Exporting a conversation

`/export` → clipboard or a plain-text file. For scripting instead: `claude -p --resume <id> --output-format json "summarize what we changed" | jq -r '.result'`.

Worktrees — isolated parallel sessions

```
claude --worktree feature-auth      # creates .claude/worktrees/feature-auth/ on
↪  branch worktree-feature-auth
```

Add `.claude/worktrees/` to `.gitignore`. Ask Claude "work in a worktree" mid-session and it uses the `EnterWorktree` tool itself.

Isolation enforcement

Once in a worktree, Claude Code **blocks**: edits targeting the main checkout, commands whose working directory resolves to the main checkout, git commands redirected into the main checkout (`git -C`, `GIT_DIR`, etc.), and any command shape it can't verify stays inside the worktree.

Copy untracked files into every new worktree

```
# .worktreeinclude (gitignore syntax)
.env
.env.local
config/secrets.json
```

Isolate subagents in their own worktree

```
---
name: refactorer
description: Applies mechanical refactors across many files
isolation: worktree
---
Apply the requested refactor, then run tests and report results.
```

Cleanup

Exiting an unnamed, clean worktree session auto-removes it. A worktree with uncommitted work prompts you first. Manually: `git worktree remove <path>` (add `--force` for uncommitted changes; `git worktree unlock` first if locked).

Checkpointing — undoing Claude's work

Before every edit, Claude Code snapshots the file. Press `Esc` twice to rewind, or just ask Claude to undo. - Checkpoints are **separate from git** and survive resuming a session. - Only cover **file changes** — not DB writes, API calls, deployments (use permissions for those). - `/rewind` → pick a message → "Summarize from/up to here" to compact only part of a conversation while keeping the rest verbatim — cheaper than a full `/compact` since it jumps to an already-cached point.

09 — Agents in Parallel (Full Comparison)

This is the direct answer to “what’s the difference between subagents, agent view, agent teams, and dynamic workflows” — four genuinely different mechanisms that all parallelize work.

The master comparison table

Approach	What it gives you	Use it when
Subagents	Delegated workers *inside one session*, own context, return a summary	A side task would flood your main conversation with stuff you won’t reference again
Agent view (`claude agents`)	One screen to dispatch/monitor many **background sessions**	Several independent tasks — hand off, check status, step in only when needed
Agent teams (experimental)	Multiple **coordinated sessions**, shared task list, message each other directly, managed by a lead	You want Claude to split a project into pieces, assign them, and keep workers in sync
Dynamic workflows	A **script** that runs many subagents and cross-checks results	A job outgrows a handful of subagents — codebase-wide audit, big migration, cross-checked research

Three supporting (not competing) mechanisms: - **Worktrees** — give each session its own git checkout so parallel edits never collide - **Cross-session messaging** — lets Claude pass findings between sessions *you* run yourself - **`/batch`** — a skill

that splits one large change into 5–30 worktree-isolated subagents, each opening its own PR

Choosing which one

Question	Answer points to
Who coordinates? Claude, inside one conversation	Subagents
Who coordinates? You hand off and check back later	Agent view
Who coordinates? Claude plans/assigns/supervises a group	Agent teams
Who coordinates? A script holds the plan, not turn-by-turn judgment	Dynamic workflows
Do workers need to talk to each other?	Agent teams (direct messaging) or cross-session messaging (sessions you run)
Do tasks touch the same files?	Isolate with worktrees; agent teams don’t auto-isolate — partition file ownership manually

Agent view — `claude agents`

One terminal screen listing every background session, grouped by state (`Needs input`, `Working`, `Completed`, etc.).

Quick start

```
claude agents                    # open the dashboard
```

Type a prompt + `Enter` to dispatch a new background session. Each prompt starts a **new** session (not a follow-up).

Key	Action
`Space`	Peek at a row's latest output/question without attaching
`Enter` / `→`	Attach — takes over the terminal as a full session
`←` (on empty prompt, inside a session)	Background the current session and open agent view
`Ctrl+X`, `Ctrl+X` again	Stop, then delete
`Ctrl+T`	Pin (keeps its process alive)

From your shell

```
claude --bg "investigate the flaky test"          # dispatch straight to background
claude --agent code-reviewer --bg "address PR comments"
claude --bg --exec 'pytest -x'                    # run a shell command as a
↪  background job, no model involved
```

From inside a session

- `/background` or `/bg` — move current conversation to background, free your terminal
- `/fork` — copy the conversation into a **new** background session while the original keeps running

File isolation

Every dispatched session moves into its own git worktree under `.claude/worktrees/` before editing — parallel background sessions never collide. When it finishes, Claude commits, pushes, and opens a PR when appropriate (never pushes to `main`, never force-pushes, never merges).

Manage from the shell

```
claude agents --json          # machine-readable session states, for
↪  scripts/dashboards
claude attach <id>
claude logs <id>
claude stop <id>
claude rm <id>
```

Agent teams (experimental, disabled by default)

```
{ "env": { "CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS": "1" } }
```

One session is the **lead**; it spawns teammates that each get their own context window and can message each other directly (not just report back).

Subagents vs Agent teams

	Subagents	Agent teams
Context	Own window, result returns to caller	Own window, fully independent
Communication	Return a result	Message each other directly
Coordination	Main agent manages everything	Self-coordinate via shared task list
Token cost	Lower	Higher — each teammate is a full Claude instance
Best for	Focused tasks, only the result matters	Complex work needing discussion/debate

Starting a team

```
I'm designing a CLI tool for tracking TODOs. Spawn three teammates to
explore this from different angles: one on UX, one on architecture,
one playing devil's advocate.
```

Claude populates a shared task list, spawns teammates, and synthesizes findings when done. Sometimes Claude spawns ordinary subagents instead — if you specifically want a team, say so explicitly.

Best practices

- Start with **3–5 teammates** — coordination overhead grows, returns diminish past that
- **Give teammates full context in the spawn prompt** — they don't inherit the lead's conversation history, only CLAUDE.md/MCP/skills
- **Partition file ownership** — teams don't auto-isolate into separate worktrees; two teammates editing the same file will conflict
- Strongest use cases: parallel code review (each teammate a different lens), competing-hypothesis debugging (teammates try to disprove each other), cross-layer features (frontend/backend/tests each owned separately)

Known limitations

No session resumption for in-process teammates after `/resume`//`rewind`; one team per session, no nested teams; a teammate can't be promoted to lead; permission mode is fixed at spawn time.

Dynamic workflows — script-based orchestration

A JavaScript script (that Claude writes) coordinates dozens to hundreds of subagents in the background, using primitives like `agent()`, `parallel()`, `pipeline()`, `phase()`. Available on paid plans; on Pro, turn on from `/config`.

Key distinction from subagents/teams: with subagents/teams/skills, *Claude* is the orchestrator — it decides turn-by-turn what to spawn, and every result enters its context. With a workflow, the **script** holds the loop, branching, and intermediate results — Claude's context only holds the final answer.

```
Use a dynamic workflow to sweep the whole codebase for SQL injection
risks, then cross-check each finding with a second pass before reporting.
```

Try the built-in example: `/deep-research` — investigates a question across many sources, phase by phase, in the background while your session stays responsive.

Use a workflow when: stage N's output determines stage N+1 (route, score, filter, loop, retry) — not just a single round of parallel judgment (that's what subagents/teams already do well).

Check progress: `/workflows` lists running/completed runs and how many agents finished in each.

Cross-session messaging

Lets one of *your own* Claude Code sessions deliver a text message to another — never conversation history or files, just text. Two tools power it: `ListAgents` (discover reachable sessions) and `SendMessage` (deliver by name).

Use it when: a change in one session breaks what another is building on, or one session settles a question blocking another. A message is delivered/held/refused based on the receiving session's own inbound controls — permission boundaries stay per-session; a teammate can never approve a permission prompt on your behalf.

10 — MCP Deep Dive

MCP (Model Context Protocol) is how Claude Code reaches outside your local files — databases, issue trackers, dashboards, browsers, your own internal tools. Connect a server the moment you catch yourself copying data into chat from another tool.

What people actually use it for

- “Add the feature described in JIRA issue ENG-4521 and create a PR on GitHub”
- “Check Sentry and Statsig for the usage of feature ENG-4521”
- “Find emails of 10 random users who used feature ENG-4521 from our Postgres database”
- React to external events — an MCP server can also push messages into your session as a **channel** (Telegram, Discord, webhooks) so Claude reacts while you’re away

  Note: Verify you trust a server before connecting — a server that fetches external content can expose you to prompt injection.

Adding a server — 4 transport types

1. Remote HTTP (recommended for cloud services)

```
claude mcp add --transport http notion https://mcp.notion.com/mcp
claude mcp add --transport http secure-api https://api.example.com/mcp \
  --header "Authorization: Bearer your-token"
```

2. Remote SSE (deprecated — use HTTP where available)

```
claude mcp add --transport sse asana https://mcp.asana.com/sse
```

3. Local stdio (runs as a process on your machine)

```
claude mcp add --env AIRTABLE_API_KEY=YOUR_KEY --transport stdio airtable \
  -- npx -y airtable-mcp-server
```

Everything after `--` is passed straight to the server — this is how Claude Code separates its own flags from the server's.

4. Remote WebSocket (servers that push events unprompted)

```
claude mcp add-json events-server \

  ↪  '{"type":"ws","url":"wss://mcp.example.com/socket","headers":{"Authorization":"Be
  ↪  TOKEN"}}'
```

Scopes — who sees the server

Scope	Loads in	Shared with team	Stored in
Local (default)	Current project only	No	`~/.claude.json`
Project	Current project only	Yes, via `.mcp.json` in repo	`.mcp.json`
User	All your projects	No	`~/.claude.json`

```
claude mcp add --transport http stripe --scope local https://mcp.stripe.com   #
 ↪  personal, this project
claude mcp add --transport http shared-server --scope project https://x.com/mcp  #
 ↪  commit .mcp.json
claude mcp add --transport http hubspot --scope user
 ↪  https://mcp.hubspot.com/anthropic  # you, everywhere
```

Project-scoped servers from `.mcp.json` **prompt for approval** the first time (a repo you cloned can't silently grant itself server access). Reset choices with `claude mcp reset-project-choices`.

Managing servers

```
claude mcp list                    # health status for every server: Connected / Needs auth / Failed
claude mcp get notion               # detail on one server
claude mcp remove notion            # also deletes stored OAuth tokens
/mcp                                # in-session panel: same info, plus toggle servers on/off
```

Authentication

Most cloud MCP servers use OAuth 2.0:

```
claude mcp add --transport http sentry https://mcp.sentry.dev/mcp
/mcp    # follow the browser login flow
```

Or from the shell directly: `claude mcp login sentry` / `claude mcp logout sentry`.

For servers needing pre-registered OAuth apps (no dynamic client registration):

```
claude mcp add --transport http --client-id YOUR_ID --client-secret --callback-port 8080 \
  my-server https://mcp.example.com/mcp
```

For non-OAuth auth (Kerberos, short-lived tokens, internal SSO) — generate headers dynamically at connection time:

```
{
  "mcpServers": {
    "internal-api": {
      "type": "http",
      "url": "https://mcp.internal.example.com",
      "headersHelper": "/opt/bin/get-mcp-auth-headers.sh"
    }
  }
}
```

Environment variable expansion in `.mcp.json`

```
{
  "mcpServers": {
    "api-server": {
      "type": "http",
      "url": "${API_BASE_URL:-https://api.example.com}/mcp",
      "headers": { "Authorization": "Bearer ${API_KEY}" }
    }
  }
}
```

`${VAR}` expands; `${VAR:-default}` falls back. Note: Claude Code's own credentials (`ANTHROPIC_API_KEY`, cloud provider tokens, etc.) always read as **empty** in a remote server's `url`/`headers` — this stops a project's `.mcp.json` from silently exfiltrating your Claude Code credentials to a third-party server.

Reference MCP resources with @-mentions

```
Can you analyze @github:issue://123 and suggest a fix?
Compare @postgres:schema://users with @docs:file://database/user-model
```

Type `@` to see everything available from connected servers, alongside your files.

MCP prompts as commands

```
/mcp__github__list_prs
/mcp__jira__create_issue login-bug high
```

Claude Code lists these under `/servername:promptname (MCP)` in the `/` menu.

Tool search — keeps context cheap as you add servers

By default, only tool **names** load at session start; full schemas load on demand when Claude actually needs them via tool search. This means adding more MCP servers has minimal context impact — there's no fixed per-server tool cap, just your overall context budget.

```
ENABLE_TOOL_SEARCH=auto:5 claude    # load tools upfront only while they total < 5%
↪  of context
ENABLE_TOOL_SEARCH=false claude     # disable deferral entirely, load everything
↪  upfront
```

Exempt a specific server that Claude needs on every turn:

```
{ "mcpServers": { "core-tools": { "type": "http", "url": "...", "alwaysLoad": true
↪  } } }
```

Output limits

- Warning at 10,000 tokens of tool output; hard default cap 25,000 tokens.
- Over the cap: Claude Code saves the result to a file and gives Claude the path instead.

```
export MAX_MCP_OUTPUT_TOKENS=50000
```

Worked examples

GitHub for code review:

```
claude mcp add --transport http github https://api.githubcopilot.com/mcp/ \
  --header "Authorization: Bearer YOUR_GITHUB_PAT"
```

```
Review PR #456 and suggest improvements
Show me all open PRs assigned to me
```

Read-only PostgreSQL querying:

```
claude mcp add --transport stdio db -- npx -y @bytebase/dbhub \
  --dsn "postgresql://readonly:pass@prod.db.com:5432/analytics"
```

```
What's our total revenue this month?
Find customers who haven't made a purchase in 90 days
```

> Use a **read-only** database user in the connection string so Claude's queries can never modify data.

Using Claude Code itself as an MCP server

```
claude mcp serve
```

Silent, blocked terminal = it's running and waiting for a client. Add to Claude Desktop's config to let it drive Claude Code's own tools from another app.

11 — Skills Deep Dive

Claude Code skills follow the open **Agent Skills standard** (works across multiple AI tools), with Claude Code extensions on top: invocation control, subagent execution, dynamic context injection.

Decision rule: when to write one

You keep pasting the same checklist into chat, or a section of CLAUDE.md has grown from a *fact* into a *procedure*. Unlike CLAUDE.md, a skill's body only loads into context when it's actually used.

Anatomy (recap + more detail)

```
.claude/skills/security-check/
├── SKILL.md
└── checklist.md          # optional supporting files — read only when mentioned
```

The directory name is what you type: `/security-check`.

A full worked example

```
---
name: summarize-changes
description: Summarizes uncommitted changes and flags anything risky. Use when the
 ↪  user asks what changed, wants a commit message, or asks to review their diff.


## Current diff

!`git diff HEAD`
```

```
Summarize the changes above in plain language. Flag anything that looks risky:
schema changes, deleted files, credentials, or config changes.
```

The `!` line before the backtick-command runs a shell command and injects its live output directly into the prompt — "dynamic context injection." This means the skill's instructions arrive already grounded in the current working-tree state, not a stale description.

Every frontmatter field

Field	Effect
`name`	The skill's identifier (defaults to the directory name)
`description`	Determines when Claude auto-invokes it — be specific about *when* to use it
`disable-model-invocation: true`	Only you can trigger it via `/name`; Claude never invokes it on its own
`user-invocable: false`	Hidden from the `/` menu — Claude can still invoke it, you can't manually
`argument-hint: <text>`	Shown in the `/` menu as a hint for expected input
`allowed-tools`	Restricts which tools the skill can use
`context: fork`	Runs the skill in an **isolated subagent context** instead of inline in your conversation

> Only 6 of these frontmatter fields are part of the open Agent Skills standard and survive packaging for claude.ai / the Skills API — the rest are Claude Code extensions. If you're sharing a skill outside Claude Code, check which fields you're relying on.

Bundled skills (ship with Claude Code, available every session)

`/doctor`, `/code-review`, `/batch`, `/debug`, `/loop`, `/claude-api` — these are **prompt-based**: they give Claude a detailed playbook and let it orchestrate using its own tools, rather than running fixed logic directly.

Bundling reference material

Large reference docs, API specs, or example collections don't need to load every time the skill runs — keep `SKILL.md` focused on the essentials and let Claude read supporting files only when it needs the detail:

```
Use checklist.md in this skill directory for the full review checklist.
```

This pattern also works for **scripts that generate visual output** — dependency graphs, coverage reports, schema diagrams. The bundled script does the mechanical work; Claude just orchestrates.

Precedence when names collide

Four storage locations, checked in this order: **managed (enterprise) → personal (`~/.claude/skills/`) → project (`.claude/skills/`) → plugin** (namespaced as `plugin-name:skill-name`, e.g. `/my-plugin:review`). A project skill wins over a personal one with the same name.

Skills vs Commands (recap)

`.claude/commands/*.md` is the older, flat-file mechanism — same `/name` invocation, no bundled supporting files. Your existing command files **keep working**; new work should use skills for the extra features (bundling, invocation control, forking).

Skills in the Agent SDK

The same skills mechanism works when you build with the Agent SDK — you control which skills an SDK session can invoke, dispatch by name, and author skills the SDK session discovers the same way an interactive session would. See file 17 for the Agent SDK section.

If your YAML frontmatter is malformed

Claude Code loads the skill body with empty metadata — `/skill-name` still works, but Claude can't auto-match against your description since there's nothing to match. Run with `--debug` to see the parse error.

12 — Hooks Reference

A hook runs your script (or an HTTP request / MCP tool call / prompt / subagent) automatically at a lifecycle event. Defined in `settings.json` — **enforced**, not guidance.

The full event list, by cadence

Once per session: | Event | Fires when | |—|—| | `SessionStart` | Session begins or resumes | | `SessionEnd` | Session ends (`reason: clear, logout, prompt_input_exit, other`) | | `Setup` | Started with `--init-only`, or `--init`/`--maintenance` in `-p` mode — for one-time CI prep |

Once per turn: | Event | Fires when | |—|—| | `UserPromptSubmit` | You submit a prompt, before Claude processes it | | `UserPromptExpansion` | A typed command expands into a prompt, before it reaches Claude — can block the expansion | | `Stop` | Main agent finishes its turn | | `StopFailure` | Main agent's turn ends in error | | `PreCompact` | Before compaction (manual or auto) |

On every tool call (the agentic loop): | Event | Fires when | |—|—| | `PreToolUse` | Before a tool call executes — **can block it** | | `PermissionRequest` | When a permission dialog appears | | `PermissionDenied` | A tool call is denied by the auto-mode classifier | | `PostToolUse` | After a tool call succeeds | | `PostToolUseFailure` | After a tool call fails | | `PostToolBatch` | After a full batch of parallel tool calls resolves | | `SubagentStart` / `SubagentStop` | A subagent starts / finishes |

Standalone / async: `Notification`, `WorktreeCreate`, `WorktreeRemove`, `ConfigChange`, `InstructionsLoaded`, `CwdChanged`, `FileChanged`, `MessageDisplay`, `TeammateIdle`, `TaskCreated`, `TaskCompleted`, `Elicitation`, `ElicitationResult`.

Exit code behavior (the part that trips people up)

Hook event	Exit code 2
`PreToolUse`	**Blocks** the tool call; stderr shown to Claude
`PostToolUse`	Can’t block (tool already ran); stderr shown to Claude as an error
`UserPromptSubmit`	Blocks prompt processing, clears the prompt; stderr to **you** only
`Stop` / `SubagentStop`	Blocks the stop, stderr shown to Claude(/subagent)
`Notification`, `PreCompact`, `SessionStart`, `SessionEnd`	No blocking; stderr shown to you only

Plain `stdout` on exit code `0` does **not** reach Claude’s context — goes to the debug log only. To actually inject content into Claude’s context, return structured JSON with `hookSpecificOutput.additionalContext`.

Common input JSON fields (every hook receives these)

```
{
  "session_id": "abc123",
  "transcript_path": "/path/to/transcript.jsonl",
  "cwd": "/current/working/dir",
  "permission_mode": "ask",
  "hook_event_name": "PreToolUse"
}
```

Event-specific additions: `PreToolUse`/`PostToolUse` add `tool_name`, `tool_input`, `tool_response`; `UserPromptSubmit` adds `prompt`; `Stop`/`SubagentStop` add `stop_hook_active` (check this to avoid an infinite stop-hook loop).

Environment variables available to command hooks

Variable	Value
`CLAUDE_PROJECT_DIR`	Project root path (stable — doesn't follow `cd` or worktree moves)
`CLAUDE_PLUGIN_ROOT`	Plugin's directory (for portable hook paths in plugins)
`CLAUDE_ENV_FILE`	`SessionStart` only — write env vars here to persist them into the session

Practical example: block Claude from reading `.env`

```
{
  "hooks": {
    "PreToolUse": [{
      "matcher": "Read|Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.file_path' | grep -q '\\.env' && exit 2 ||
        ↪  exit 0"
      }]
    }]
  }
}
```

Practical example: auto-format after every edit

```
{
  "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.file_path' | xargs npx prettier --write"
      }]
    }]
  }
}
```

Re-inject context after compaction

A `SessionStart` hook that matches the `compact` source can restore context that would otherwise be lost — the source is set to `"resume"` or `"fork"` on resume so hooks can refresh stale values like timestamps or commit SHAs (values captured before a `--continue`/`--resume` are replayed verbatim, not re-run).

Enforcing quality gates on agent teams

Three hooks specifically for agent-team coordination (see file 09): - `TeammateIdle` — exit 2 to send feedback and keep a teammate working instead of going idle - `TaskCreated` — exit 2 to prevent creation and send feedback - `TaskCompleted` — exit 2 to prevent marking complete and send feedback

Hooks vs Skills — which one

Something must happen **every single time**, no room for Claude to skip it → hook. Something Claude should *know how to do* but you’re fine with it deciding when → skill.

13 — Permissions & Security

The golden rule (worth repeating)

Permission rules are enforced by **Claude Code**, not the model. Your prompt or CLAUDE.md can shape what Claude *tries* to do — it can't change what Claude Code actually *allows*.

Rule syntax

Every rule is a string: `ToolName` or `ToolName(content)`.

```
{
  "permissions": {
    "allow": ["Bash(npm run *)", "Bash(git commit *)", "Edit(/src/**)"],
    "deny":  ["Bash(git push *)", "Read(./.env)"],
    "ask":   []
  }
}
```

- `Bash` alone (bare tool name) removes the **entire tool** from Claude's context — Claude never even sees it exists.
- `Bash(rm *)` (scoped) leaves the tool available but blocks matching calls when attempted.
- File-tool paths use gitignore-style glob patterns. `Edit(/src/)` in project settings is anchored to the project root; `Edit(src/)` (no leading slash) matches `src/` at any depth below the current directory.

Evaluation order — this is the part people get wrong

deny → ask → allow. The first match in that order wins, and **specificity never overrides order**:

```
{ "permissions": { "allow": ["Bash(git *)"], "deny": ["Bash(git push *)"] } }
```

`git commit -m "fix"` passes. `git push origin main` is **blocked**, even though a broader `allow` rule also technically matches — a narrower allow can never carve an exception out of a broader deny.

> **Note: Read/Edit deny rules only block Claude's built-in file tools — not Bash subprocesses.** A `Read(./.env)` deny rule blocks the `Read` tool, but doesn't stop `cat .env` run via Bash. Protect secrets at the OS/repo layer too; a permission rule should never be your only control around a credential.

Where rules live (4 scopes)

File	Scope
`~/.claude/settings.json`	Your personal default, every project
`.claude/settings.json`	Project, committed, shared with team
`.claude/settings.local.json`	Your personal override for this project, auto-gitignored
Managed policy settings	Organization-enforced, can't be overridden

Permission modes (recap + detail)

Cycle with `Shift+Tab`: | Mode | Behavior | |—|—| | **Auto** | Classifier reviews in background, blocks only risky actions (default on Pro/Max/Team) | | **Manual** | Asks before every edit and shell command | | **Accept edits** | Edits + safe filesystem commands run without asking | | **Plan** | Explores and proposes a plan — no editing | | **dontAsk** | Never prompts — a matching `ask` rule is **denied** instead (since there's no one to ask) | | **bypassPermissions** | Approves everything — requires one-time interactive acceptance of a disclaimer first |

Set the starting mode: `permissions.defaultMode` in settings.

Matching a specific parameter, not just the tool

```
{ "permissions": { "ask": ["Bash(command:rm)"] } }
```

Deny/ask rules can match a top-level input parameter on any built-in tool with `Tool(param:value)`.

Sandboxing (brief)

Claude Code's built-in **sandboxed Bash tool** provides filesystem and network isolation so risky commands can run more autonomously without a permission prompt for every one. Compare against dev containers, Docker, or VMs depending on your threat model — the sandboxed Bash tool is the lightest-weight option, a full VM the heaviest but most isolated.

Security basics worth internalizing

- **Prompt injection**: content Claude reads (a web page, an MCP tool's output, a file) can contain instructions trying to redirect Claude's behavior. Be cautious connecting MCP servers you don't trust, especially ones that fetch external content.
- **Workspace trust**: the first time you run `claude` in a new directory, you get a trust dialog — accepting it is what allows hooks and headersHelper scripts in that project's settings files to actually run. A cloned repo's own committed settings can't silently grant itself trust.
- **`bypassPermissions` requires one-time acceptance**: you must interactively accept a disclaimer once before this mode becomes available at all, since it lets a session you might not be watching act with zero approval.

14 — Settings, CLI & Commands Reference

Settings precedence (highest wins)

```
CLI flags / --settings payload
        ↓
Managed policy settings (organization-enforced)
        ↓
.claude/settings.local.json   (personal, this project)
        ↓
.claude/settings.json         (team, this project)
        ↓
~/.claude/settings.json       (personal, every project)
```

Array settings (like `permissions.allow`) **combine** across every scope that sets them. Scalar settings (like `model`) use the single most-specific value.

A realistic full settings.json

```
{
  "model": "claude-sonnet-4-6",
  "permissions": {
    "allow": ["Read", "Write", "Bash(git *)"],
    "deny": ["Read(./.env)", "Read(./.env.*)", "Write(./production.config.*)"]
  },
  "hooks": {
    "PostToolUse": [{
      "matcher": "Write(*.py)",
      "hooks": [{ "type": "command", "command": "python -m black $file" }]
    }]
```

```
  }
}
```

CLI flags worth knowing

Flag	Does
`-p, --print "<prompt>"`	Non-interactive — answer once and exit (great for scripts/CI)
`-c, --continue`	Reopen the most recent conversation here
`-r, --resume [name/id]`	Resume a specific or picked session
`-n, --name <name>`	Give the session a name at launch
`--model <name>`	Set the model for this session
`--permission-mode <mode>`	Start in a specific permission mode
`--dangerously-skip-permissions`	Skip all prompts (sandboxed/CI use only)
`--add-dir <path>`	Grant access to an extra directory
`--settings <file-or-json>`	Override settings for this run
`--mcp-config <file-or-json>`	Load specific MCP servers
`--strict-mcp-config`	Use *only* the servers passed via `--mcp-config`
`--worktree <name>` (or `-w`)	Start in an isolated git worktree
`--bg` / `--background`	Dispatch straight to a background session
`--output-format json|stream-json`	Structured output for `-p` scripting
`--append-system-prompt <text>`	Add to the default system prompt without replacing it
`--system-prompt <text>`	Replace the default system prompt entirely (you own the consequences)

`claude --help` doesn't list every flag — absence from `--help` doesn't mean unavailable.

Built-in slash commands (grouped)

Session & context | Command | Does | |—|—| | `/clear` | Fresh empty context (old conversation still resumable) | | `/compact [focus]` | Summarize history to free space

| | `/context` | Live breakdown of what's using your context | | `/rewind` | Jump back to an earlier turn | | `/recap` | Terminal-only summary, doesn't touch the cache | | `/resume`, `/branch`, `/rename` | Session management (see file 08) | | `/export` | Copy or save the conversation as text |

Config & setup | Command | Does | |—|—| | `/init` | Generate a starter CLAUDE.md | | `/doctor` | Setup checkup + CLAUDE.md trim suggestions | | `/memory` | Open/edit CLAUDE.md and auto memory files | | `/config` | Set any setting from the prompt | | `/permissions` | View/edit permission rules | | `/model`, `/effort`, `/fast` | Model, effort level, fast-mode toggles | | `/theme`, `/vim`, `/terminal-setup` | Interface customization |

Work & review | Command | Does | |—|—| | `/code-review` | Multi-agent PR/diff review | | `/batch` | Split one large change into many worktree-isolated subagents, each opening a PR | | `/debug` | Diagnostic debugging skill | | `/loop` | Run a prompt repeatedly / poll for status | | `/goal` | Keep working until a completion condition is met | | `/workflows` | List/save dynamic workflow runs | | `/agents` (note: doesn't open a panel — prints subagent file locations) | | | `/tasks` | Everything running in the background of the current session | | `/mcp` | MCP server status/auth panel | | `/usage`, `/cost` | Token usage and spend for this session | | `/status` | Current session info | | `/bug` | Report a bug to Anthropic |

Environment variables worth knowing

Variable	Effect
`ANTHROPIC_API_KEY`	API key auth (bypasses claude.ai login)
`CLAUDE_CODE_PROMPT_CACHE_TTL`	`5m` or `1h` — main conversation cache lifetime
`CLAUDE_CONFIG_DIR`	Move `~/.claude` storage elsewhere
`DISABLE_AUTOUPDATER`	Set `1` to control when Claude Code upgrades apply
`MAX_MCP_OUTPUT_TOKENS`	Raise the MCP tool output cap (default 25,000)
`ENABLE_TOOL_SEARCH`	`true`/`false`/`auto:N` — control MCP tool deferral
`CLAUDE_CODE_SKIP_PROMPT_HISTORY`	Suppress transcript/history writes entirely
`CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS`	`1` to enable agent teams

Where Claude Code stores things (recap)

`~/.claude/projects/<project>/<session>.jsonl` — full transcripts, plaintext, unencrypted. `~/.claude/projects/<project>/memory/` — auto memory (never auto-swept). Lower `cleanupPeriodDays` in settings if you want shorter transcript retention.

15 — Model Config & Costs

Choosing a model

Model	Best for
Sonnet	Day-to-day: writing tests, simple edits, explaining code, refactoring
Opus	Complex: multi-file architecture decisions, gnarly cross-system debugging
Haiku	Quick/mechanical: lookups, formatting, renaming, repetitive tasks

```
/model sonnet   # switch mid-session
/model opus
```

On the API, Opus costs roughly **5x** Sonnet per token — start every session on Sonnet, escalate to Opus only when you genuinely need the deeper reasoning.

Effort levels

On models with adaptive reasoning, **effort level** (not just model choice) is a primary lever for cost and depth:

```
/effort low
/effort high
```

Lowering effort on a straightforward task directly reduces the thinking-token budget — meaningful savings on mechanical work.

Context windows

- **Sonnet 5** (current default) and the Fable models run a **native 1M token context window** — no `[1m]` variant to select.
- Older/other configurations (Sonnet 4.6, Opus 4.6 without extended context, or any model on Bedrock/Vertex/Foundry running at 200K) compact at the 200K boundary.
- Models with a genuine 1M window compact **before** the window fully fills — around 967K tokens by default, not at the hard limit.

Set your own compaction threshold:

```
/autocompact 500k
```

Fast mode

Trades a request-header change for faster Opus responses. Turning it on costs one uncached turn (see file 07 on caching) — turning it on **at the start** of a session is cheaper than deep into a long one, since the cache-miss cost is the same either way but you've accumulated more conversation by then.

The advisor tool

Pairs your main model with a **stronger advisor model** that Claude consults at key decision points during a task — a way to get occasional Opus-level judgment without running the whole session on Opus.

Managing costs

The numbers (enterprise average, for context)

Around **$13/developer/active-day**, **$150–250/developer/month**; 90% of users stay under $30/active-day. Unexpectedly high spend usually traces to: long sessions that were never `/clear`ed, or Opus left as the default model for everything.

Reduce token usage — the concrete levers

1. **CLAUDE.md is the highest-leverage cost lever.** A well-written 300–500 word file eliminates 500–2,000 tokens of repeated project explanation *every single session*. Write it once, it pays back every time.
2. `/clear` **between unrelated tasks.** Stale context wastes tokens on every subsequent message, not just the current one.
3. **Match model to task.** Sonnet for daily work, Opus only for genuinely hard problems, Haiku for mechanical/repetitive work.
4. **Be specific in prompts** so Claude reads fewer files ("fix the bug in auth.ts" beats "fix the bug").
5. **Delegate research to subagents** — large file reads stay in the subagent's isolated context, you only pay for the summary.
6. **Check usage proactively**: `/usage` for a session breakdown, or configure your statusline to show it continuously.

Agent team costs specifically (see file 09)

- Use Sonnet for teammates — balances capability/cost for coordination tasks
- Keep teams small — token cost is roughly proportional to team size
- Keep spawn prompts focused — teammates load CLAUDE.md/MCP/skills automatically, but everything else you type adds to their context from the start
- Shut teammates down when done — an idle teammate still consumes tokens until it exits

16 — Tools, Plugins & Platform Integrations

Built-in tools (beyond the 5 categories in file 02)

Every built-in tool has its own permission behavior — some ask by default, some don't, some can be scoped with `Tool(param:value)` rules (see file 13):

Tool	Does
`Read`, `Write`, `Edit`, `NotebookEdit`	File operations
`Bash`, `PowerShell`	Shell execution
`Grep`, `Glob`	Search
`WebSearch`, `WebFetch`	Web access
`Task` (Agent tool)	Spawns subagents/teammates
`TaskCreate`, `TaskGet`, `TaskList`, `TaskUpdate`	Shared task list (agent teams)
`SendMessage`, `ListAgents`	Cross-session/teammate messaging
`EnterWorktree`, `ExitWorktree`	Worktree navigation
`ToolSearch`	On-demand MCP tool schema loading
`Monitor`	Long-running background watch (e.g. waiting on CI)
`WaitForMcpServers`	Used instead of tool search when tool search is unavailable

Deny a tool entirely with a bare name (`"deny": ["Bash"]`) to remove it from Claude's context altogether — see file 13 for the full permission rule mechanics.

Plugins — bundling everything into one installable unit

A plugin packages skills, subagents, hooks, and MCP servers together so you can install/share the whole set at once.

```
/plugin marketplace add anthropics/claude-plugins-official
/plugin install mcp-server-dev@claude-plugins-official
```

Manage: `/plugin`, `/reload-plugins` (needed after enabling/disabling mid-session — warns first if it would force a full context re-read; add `--force` to proceed anyway).

Plugin-bundled skills are **namespaced**: `/my-plugin:review` rather than a bare `/review`, so multiple plugins can coexist without name collisions.

Discovering plugins

Browse marketplaces the same way you'd browse an app store for Claude Code extensions — reviewed, installable in one command. Building your own: a plugin declares its components (skills/agents/hooks/MCP servers) in a manifest; test it with `claude plugin eval` against real prompts before shipping.

Platform integrations (condensed reference)

Integration	What it does
VS Code extension	Inline diffs, @-mentions, plan review, conversation history in-editor
JetBrains plugin	Same for IntelliJ/PyCharm/WebStorm — requires the CLI installed separately
Claude in Chrome	Connects to your browser — test web apps, debug via console logs, automate form filling, extract page data
Claude Code GitHub Actions	Respond to `@claude` mentions in issues/PRs, automate tasks, turn issues into PRs directly from CI
GitLab CI/CD	Equivalent integration for GitLab pipelines
Code Review (`/code-review`)	Automated multi-agent PR review — logic errors, security, regressions, full-codebase context
Remote Control	Continue a local session from your phone or any browser
Claude Code on mobile	Start/monitor/steer Claude Code tasks from the iOS/Android app

Integration	What it does
Claude Code on the web	Run sessions in the cloud, no local setup — connect a GitHub repo, submit a task, review the resulting PR
Claude Tag (Slack)	Bring Claude into team Slack channels, tag it into any conversation to delegate a task

> These platform-specific pages weren't fetched in full depth for this reference set — if you adopt one heavily (e.g. GitHub Actions for CI), it's worth reading `code.claude.com/docs/en/<page>` directly for the current setup steps, since these integrations evolve quickly.

Glossary — terms worth knowing cold

Term	Meaning
Agentic loop	gather context → act → verify → repeat, the core execution pattern
Compaction	Replacing conversation history with a structured summary to free context
Session	One saved conversation tied to a project directory
Subagent	An isolated worker spawned inside a session, own context, returns a summary
Agent team	Multiple independent sessions coordinated by a lead, messaging directly
Worktree	A separate git working directory sharing the same repo history — enables safe parallel edits
Hook	A script/action that fires automatically at a lifecycle event, enforced regardless of Claude's choices
Skill	A markdown file (+ optional bundled files) of reusable instructions, invocable or auto-triggered
MCP (Model Context Protocol)	The open standard for connecting Claude Code to external tools/services

Term	Meaning
Auto memory	Notes Claude writes about you/the project automatically, without being asked
Prompt cache	Server-side reuse of unchanged request prefix to avoid reprocessing every turn
Permission mode	The global stance controlling how much Claude can do without asking (manual/auto/plan/etc.)
Checkpoint	An automatic pre-edit file snapshot enabling `Esc Esc` rewind

17 — Best Practices & the Agent SDK

Best practices (condensed)

- **Delegate, don't dictate** — give context + goal, let Claude find the files/commands (file 02)
- **Keep CLAUDE.md under 200 lines**, push detail into skills/path-scoped rules (file 03, 08)
- `/clear` **between unrelated tasks**, `/compact` at natural breakpoints, not mid-task (files 07, 15)
- **Be specific in prompts** — "fix the bug in auth.ts" beats "fix the bug," fewer files get read
- **Use subagents for research-heavy side tasks** so large reads don't pollute your main context (file 05)
- **Pre-approve safe repeated commands** (`npm test`, `git status`) in settings so Claude stops asking (file 13)
- **Verify before connecting an MCP server** — untrusted servers are a prompt-injection vector (file 10, 13)
- **Match model and effort to the task** — don't run everything on Opus/max effort (file 15)
- **Scale across parallel sessions deliberately** — worktrees for file isolation, subagents for research, agent teams only when tasks genuinely need discussion/debate (file 09)

Common workflow patterns (from Anthropic's own guide)

- **Explore, then plan, then code** — use plan mode for anything non-trivial; a reviewed plan beats an unreviewed one-shot attempt
- **Bug fix loop**: reproduce → trace root cause → fix → verify with tests, not just "looks right"
- **Refactor loop**: read broadly first (or delegate to a subagent), then edit narrowly

- **Test-writing**: point Claude at the specific untested module rather than "write more tests"

Large codebases / monorepos

- Nested `CLAUDE.md` files per package/directory, loaded only when Claude works in that subtree
- `claudeMdExcludes` to skip other teams' CLAUDE.md files that aren't relevant to your work
- Per-package skills instead of one giant project-root skill set
- Code intelligence plugins matter more here — grep-based search gets slow at scale; a language server gives Claude symbol-level navigation instead

The Agent SDK — building your own products with Claude Code's engine

This is the part most relevant to your actual job: everything above is about using Claude Code *interactively*. The **Agent SDK** is how you embed the same agentic loop, tools, and context management into **your own application** — a production pipeline, not a terminal session.

> Available as a library for Python and TypeScript only. To drive the same loop from another language, run the CLI as a subprocess with `-p --output-format json`.

Install

```
pip install claude-agent-sdk          # Python
npm install @anthropic-ai/claude-agent-sdk   # TypeScript
```

Minimal working example (Python)

```python
import asyncio
from claude_agent_sdk import query, ClaudeAgentOptions

async def main():
    async for message in query(
        prompt="Find and fix the bug in auth.py",
        options=ClaudeAgentOptions(
            allowed_tools=["Read", "Edit", "Bash"],
            permission_mode="acceptEdits",
        ),
    ):
        if hasattr(message, "result"):
            print(message.result)

asyncio.run(main())
```

Minimal working example (TypeScript)

```typescript
import { query } from "@anthropic-ai/claude-agent-sdk";

for await (const message of query({
  prompt: "Find and fix the bug in auth.py",
  options: { allowedTools: ["Read", "Edit", "Bash"] },
})) {
  console.log(message);
}
```

`query()` returns an **async iterator** — each iteration yields a message: Claude's reasoning, a tool call, a tool result, or the final outcome. The SDK handles orchestration (tool execution, context management, retries); you just consume the stream. This is the same "agent loop" concept from file 02, just callable from your own code instead of a terminal.

What carries over from Claude Code automatically

The SDK can load the same CLAUDE.md, skills, hooks, and MCP servers your interactive sessions use — you opt into each via configuration rather than reimplementing them.

Key building blocks

Concept	What it gives you
`ClaudeAgentOptions` / config object	Model, `allowed_tools`, `permission_mode`, working directory, system prompt strategy
Custom tools	Define your own functions Claude can call (`@tool` decorator in Python, `createSdkMcpServer` in TypeScript) — an in-process MCP server for domain-specific operations
Subagents in the SDK	Same isolation concept as file 05, invoked programmatically
Hooks in the SDK	In-process callbacks — e.g. TypeScript: `options.hooks = { PostToolUse: [{ matcher: "Edit|Write", hooks: [callbackFn] }] }`
Sessions	Persist and resume agent conversation history the same way interactive sessions do
Structured outputs	Get validated JSON back from a multi-turn agent run via JSON Schema, Zod, or Pydantic
`canUseTool` callback	Build your own approval UI — this is what receives permission requests instead of a terminal prompt
Cost tracking	Token usage, cost estimation, and prompt caching configuration for your own billing/monitoring
Observability	Export traces/metrics/events via OpenTelemetry

Permission evaluation order in the SDK (same spirit as file 13, SDK-specific)

1. **Hooks** run first — can deny outright
2. **Deny rules** (from `disallowed_tools` and settings.json) — block even in `bypassPermissions`
3. **Ask rules** — fall through to your `canUseTool` callback for confirmation

4. **Permission mode** — `bypassPermissions` approves everything reaching this step; `plan` mode routes writes to your callback regardless of allow rules
5. **Allow rules** — approve if matched
6. **`canUseTool` callback** — final fallback if nothing above resolved it

> **Note:** In `bypassPermissions` mode, **subagents inherit it too** and can't be overridden individually — they get full autonomous access with zero approval prompts. Be deliberate about when you enable this in a production agent.

Error handling (Python)

```
from claude_agent_sdk import (
    ClaudeSDKError, CLINotFoundError, CLIConnectionError,
    ProcessError, ResultError, CLIJSONDecodeError,
)

try:
    async for message in query(prompt="Hello"):
        pass
except CLINotFoundError:
    print("Please install Claude Code")
except ResultError as e:
    print(f"Run failed: {e.terminal_reason=}")
except ProcessError as e:
    print(f"Process failed with exit code: {e.exit_code}")
```

Where to go deeper

The Agent SDK docs are extensive (agent loop internals, streaming input/output modes, tool search at scale, plugins in the SDK, hosting in production with Docker/Kubernetes, multi-tenant isolation). This reference set gives you the on-ramp; once you're actually building a specific agent for one of your projects, pull the exact SDK page for that feature — the API surface (especially TypeScript/Python reference pages) changes fast enough that a static copy here would go stale quickly.